High-Level Design (HLD)

Version: 1.0

Date: 16-03-2026

Prepared by: Karthika V T

1. Purpose

👉 The High Level Design document serves as a blueprint for the Fuel Pressure and Alarm system. The design document is created based on the system requirements. Based on the requirements(monitor fuel pressure, triggers alarms) the system design is implemented. The system is intended to continuously monitor fuel pressure levels in real time, detecting under pressure and over pressure.

2. System Overview

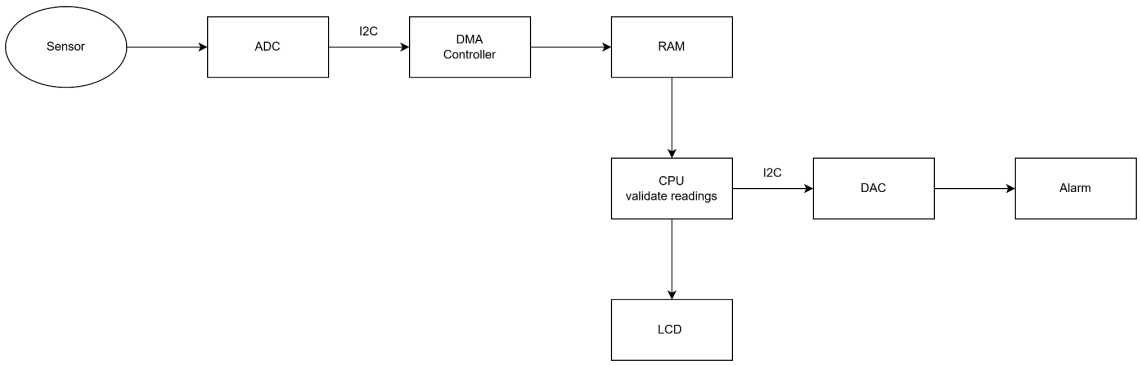
👉 Overview

The Fuel Pressure and Alarm system is designed to continuously monitor fuel pressure in real time, detect abnormal conditions and trigger alarms. It integrates sensors, processing logic and alert mechanisms to protect equipment from damages.

👉 Key Components

The key components used are fuel pressure sensors module, central processing unit, buzzer for alarm module and display unit. The sensor, display and alarm system communicates with the MCU using a standard communication protocol (I2C, SPI).

👉 High Level Architecture Diagram



3. Software Architecture

👉 Major modules

The major modules used are fuel pressure sensor module, MCU, alarm system and display unit. The fuel pressure sensor captures real time fuel pressure data and sends it to the microcontroller. MCU is the core processing unit that reads sensor data, compares it against thresholds and controls alarm and display. The alarm system activates the buzzer when pressure is outside the safe limits. The display unit shows live pressure readings and system status to the operator. The pressure sensor continuously sends analog or digital signals to the microcontroller. The data is processed by the microcontroller. If the pressure exceeds or drops below a predefined threshold, the microcontroller sends a signal to the activated alarms. The microcontroller updates the display with current pressure readings and system status.

👉 Data flow

The sensor generates a continuous analog voltage. The ADC samples this voltage and quantizes it. It transitions from an electrical signal to a hexadecimal serial bitstream. Data is sent over the I2C bus. The ADC acts as the Slave and the CPU acts as the Master. If the DMA is enabled, the DMA controller listens to the I2C bus. The DMA converts the serial bits into parallel integers to match the system bus width (AHB/APB). The sensor data will be stored like in a struct format. The CPU runs logic to convert the numeric struct data into a readable format. A bool parameter in the struct is to represent whether the value is within safe range or not, if it returns false, the CPU sends a command to the DAC of the alarm system.

👉 Control flow

The control flow of this system is governed by a hardware triggered interrupt model. The process starts with the CPU configuring the internal registers of the peripherals. The ADC is configured by the sampling rate and resolution. The DMA is configured by the source address and the destination address. The CPU loads the threshold values into its local memory to use for data validation. Once started the CPU goes into a low power state or handles other tasks while the hardware takes over the trigger, data movement and interrupt. The DMA controller detects the data ready flag from ADC. Once the DMA has filled the designated buffer in RAM, it sends an interrupt request to the CPU. The CPU pulls the data struct from RAM into its internal register. It compares the raw value against the pre-set minimum or maximum threshold. Based on the validation results, the control flow splits into two paths. In the continuous path, the CPU formats the sensor data into an ASCII string and sends it to the LCD display interface so the user can see the current reading. If the status is invalid, the CPU sends a control signal through I2C to the DAC. The DAC immediately converts this to a voltage that triggers the Alarm.

4. Module Descriptions

👉 Sensor Interface Module

It is used to manage the raw communication between the physical fuel pressure sensor and the MCU. It ensures the incoming signal is captured accurately and consistently and converts raw voltage or binary values into meaningful units. The module contains pressure sensor and

Key Interfaces - Input: Raw electrical signal from the hardware pins.

Output: Normalized digital pressure value sent to the Processing Logic.

👉 Processing and Decision Control Module

This is the main processing unit of the system. Its incoming data which in the struct format is stored in RAM using DMA is taken by the CPU to process. The pressure value taken from the struct is given to the I2C output register which is connected to the display unit. The same value is compared with the predefined threshold values. If the value is deviated from the safe range the I2C output register for the alarm system is fed with the value. The register of the alarm system is fed with the value to generate the alarm signal.

Key Interfaces - Input: Scaled pressure data from the sensor module.

Output: Status flags and command signals sent to the output drivers.

👉 Alarm Module

This module toggles the output pins connected to the buzzer hardware. It controls the frequency or pulse of the alarm. The alarm can be silenced or reset once the pressure returns to a safe range.

Key Functions - Formats text, numbers for the specific display hardware. Update the screen at a readable frequency.

Key Interfaces - Input: Current pressure values and system state flags

Output: Data packets sent over I2C to the display hardware.

5. Interface Design

👉 Application Programming Interface

Sensor APIs:

initPressure() to configure the sensor module and enable it.

readPressure() to calculate the pressure value from the input signal

Alarm control APIs:

initBuzzer() to configure the buzzer and start the operation

triggerBuzzer() output registers of the buzzer are enabled , resulting in triggering an alarm.

reserAlarm() Alarm can be reset once the user is acknowledged.

returnStatus() Alarm status can be returned for verification.

Display APIs:

initDisplay() the display is configured and enabled

updateDisplay(value, status, length) the data sent is displayed on the LCD

clearDisplay() used to clear the LCD

👉 Inter-Process Communication Methods

Shared memory with Synchronization:

Global variables are defined in shared memory that can be accessed by all threads and the access can be protected with mutexes or semaphores to avoid race conditions.

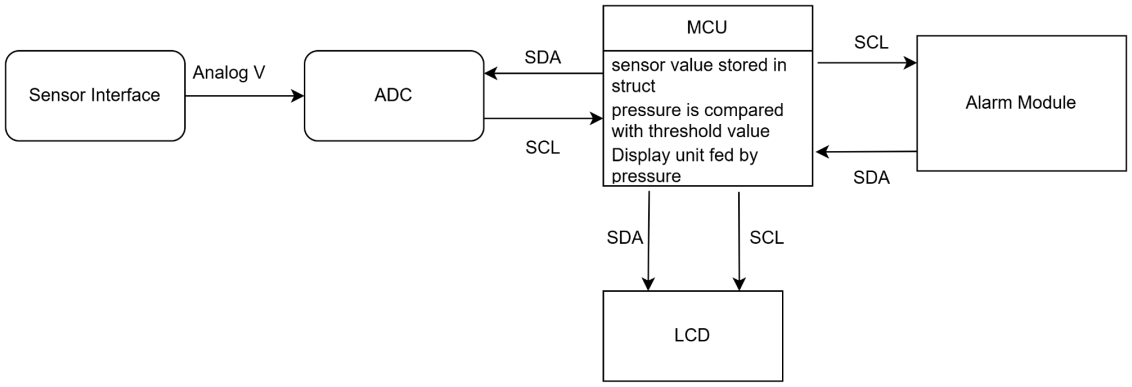
Messagequeues is used to share data between multiple threads. For synchronization mutex and semaphores can be used.

👉 Data Exchange Formats

On each phase data is formatted such that all modules can interpret it. Binary data is used between the sensor and MCU. In MCU data is stored as struct which can contain the voltage, current, status of the register and time like parameters. If the system has a WIFi module to send data to a remote app, it might wrap the pressure readings in JSON. To send data using any standard protocols like I2C specific data format is used.

6. Data Flow

👉



7. Error Handling & Fault Management

👉 Fault detection

The system continuously monitors signals and module outputs to identify abnormal conditions. Sensor faults are found when detects missing, stuck, or out of range values. Communication faults can be identified by I2C bus errors or corrupted packets. Display faults are detected when updates fail or invalid characters are printed on LCD.

👉 Fault detection

Once a fault is detected, the system isolates the source to prevent cascading failures. Sensors can be isolated by disabling the particular register of the sensor. SIgnal can be validated by cross checking sensor readings against expected ranges.

👉 Fault recovery

The system attempts to restore functionality or maintain safe operation. The system can be automatically recovered by reading the sensor after short delays or reinitializing communication buses. For manual recovery operators can acknowledge the alarm. Logs provide fault history for troubleshooting.

8. Traceability

👉 Fuel Pressure sensor interface - It acquires real time fuel pressure readings, converts analog signals to digital values and provide calibrated readings to processing unit.

Threshold Evaluation value - It compares sensor readings against predefined safe limits, detects abnormal conditions like over pressure and under pressure and generates status flags for downstream modules.

Alarm Control Module - It triggers visual alarms when unsafe conditions are detected and ensures alarms activate within defined response time.

Display Module - It displays current fuel pressure readings to operators, shows system status(NORMAL, HIGH, LOW)